

小红书大规模指标监控优化

如何提升 10 倍查询速度 节约万核 CPU

章正中

小红书可观测技术组

2025/11/15





章正中

小红书可观测技术工程师

熟悉可观测、性能分析、云原生等领域

CONTENT

01 背景与挑战

02 演进历程

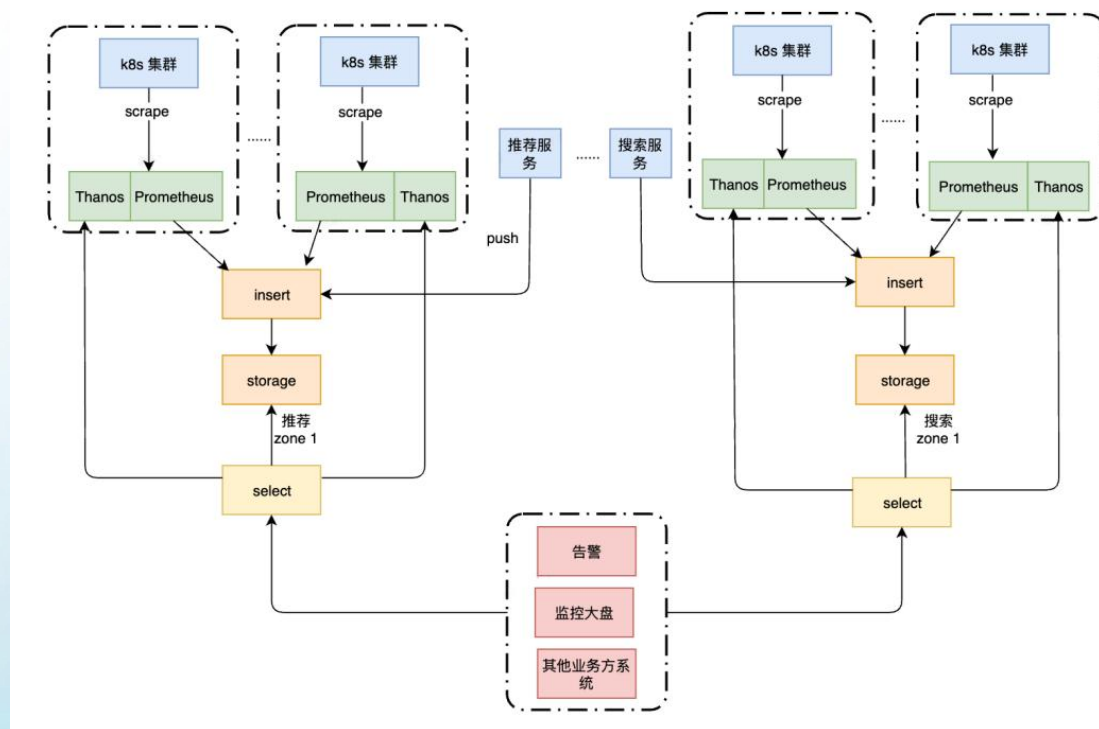
03 总结与展望



背景与挑战

旧部署架构:

- Prometheus 随业务分散部署
- VictoriaMetrics 作为长期存储
- Thanos 提供备用应急查询



指标监控链路部署架构(重构前)

旧部署架构遇到的问题

- 资源成本高
 - Prometheus 内存占用高，实例规格大，集群多
- 稳定性较差
 - 故障和变更易引发数据异常与告警误报
- 运维成本高
 - Prometheus 部署分散，无法集中管理
 - 扩缩容与配置更新流程复杂，依赖大量黑屏命令行操作
- 使用体验差
 - 查询超时

演进历程

- ❑ 采集端重构
- ❑ 高可用改造
- ❑ 高基数治理
- ❑ 跨云多活
- ❑ 查询优化

采集端重构

问题梳理

- 资源成本高，内存需求大，告警频繁
- 部署分散，运维成本高
- 配置管理繁琐，流程不规范

告警 <环境: prod> 业务prometheus 内存使用率>85% 来源: reddog 级别: P0 服务树: devops.monitor.prometheus 指标: { "value": "87.261358",	告警 <环境: prod> 业务prometheus 内存使用率>85% 来源: reddog 级别: P0 服务树: devops.monitor.prometheus 指标: { "value": "89.703144",
告警 <环境: prod> 业务prometheus 内存使用率>85% 来源: reddog 级别: P0 服务树: devops.monitor.prometheus 指标: { "value": "92.013885",	告警 <环境: prod> 非业务prometheus 内存使用率>85% 来源: reddog 级别: P1→P0 服务树: devops.monitor.prometheus 指标: { "value": "94.875592",

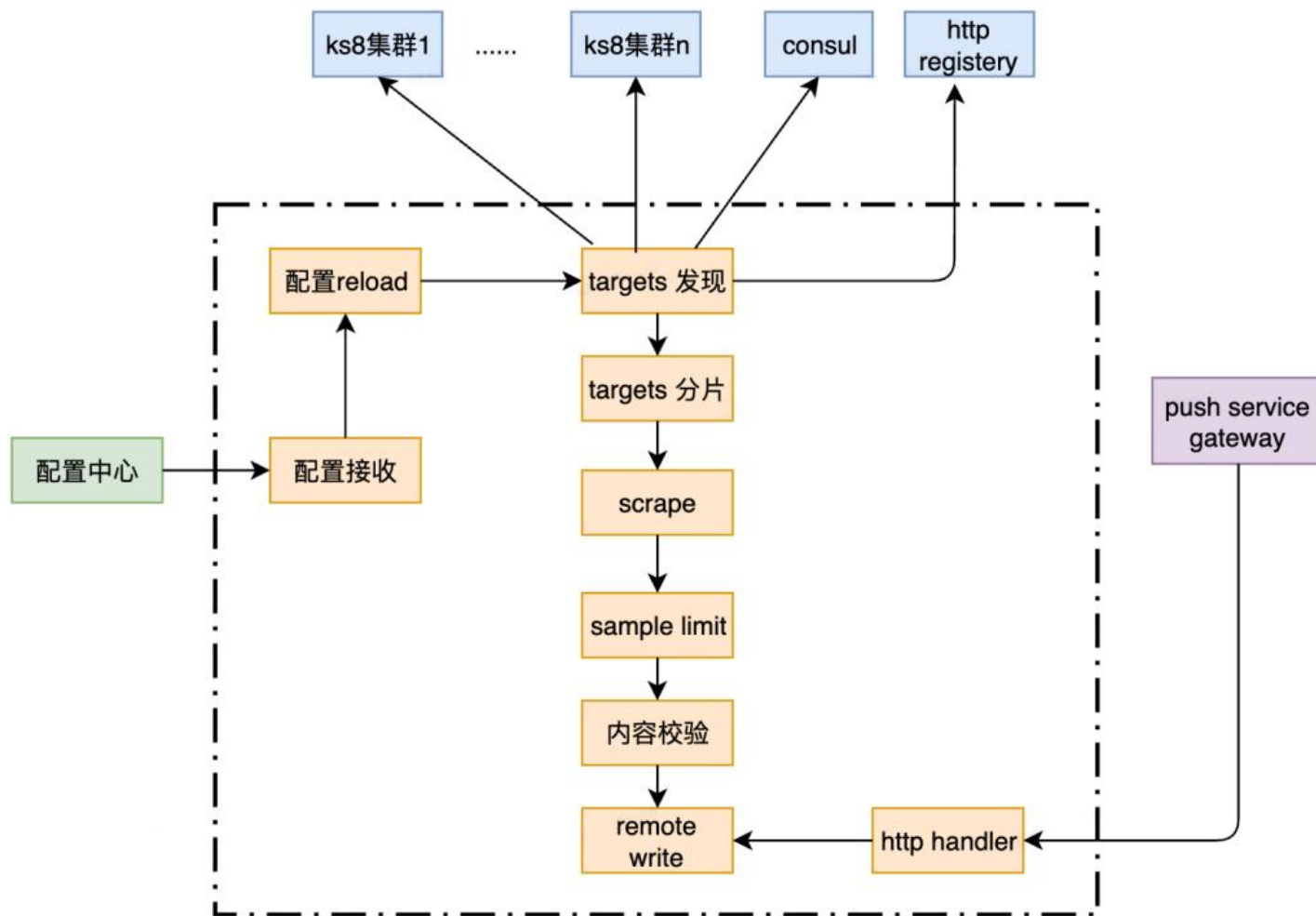
频繁内存告警

prometheus-	lt-0	3/3	Running	0	168d
prometheus-	lt-1	3/3	Running	0	133d
prometheus-	hs-default-0	2/2	Running	0	5d3h
prometheus-	ult-0	3/3	Running	0	61d
prometheus-	ult-1	3/3	Running	0	61d
prometheus-	efault-0	3/3	Running	0	177d
prometheus-		2/2	Running	0	6d3h
prometheus-	aunt-0	3/3	Running	0	191d
prometheus-	t-0	3/3	Running	0	22d
prometheus-	-default-0	3/3	Running	0	49d
prometheus-	-default-1	3/3	Running	0	49d
prometheus-	-default-2	3/3	Running	0	49d
prometheus-	-default-3	3/3	Running	0	49d
prometheus-	-default-4	3/3	Running	0	49d
prometheus-	-default-5	3/3	Running	0	49d
prometheus-	-default-6	3/3	Running	0	49d
prometheus-	-default-7	3/3	Running	0	49d
prometheus-	fault-0	3/3	Running	0	4d20h
prometheus-	lt-0	3/3	Running	0	150d
prometheus-	sh-default-0	3/3	Running	0	184d
prometheus-	aunt-0	3/3	Running	0	191d
prometheus;	-hash-0	3/3	Running	0	89d
prometheus;	-hash-1	3/3	Running	3	89d
prometheus;	-hash-2	3/3	Running	0	89d
prometheus;	-hash-3	3/3	Running	0	89d

实例数多，分散在多个集群

重构过程

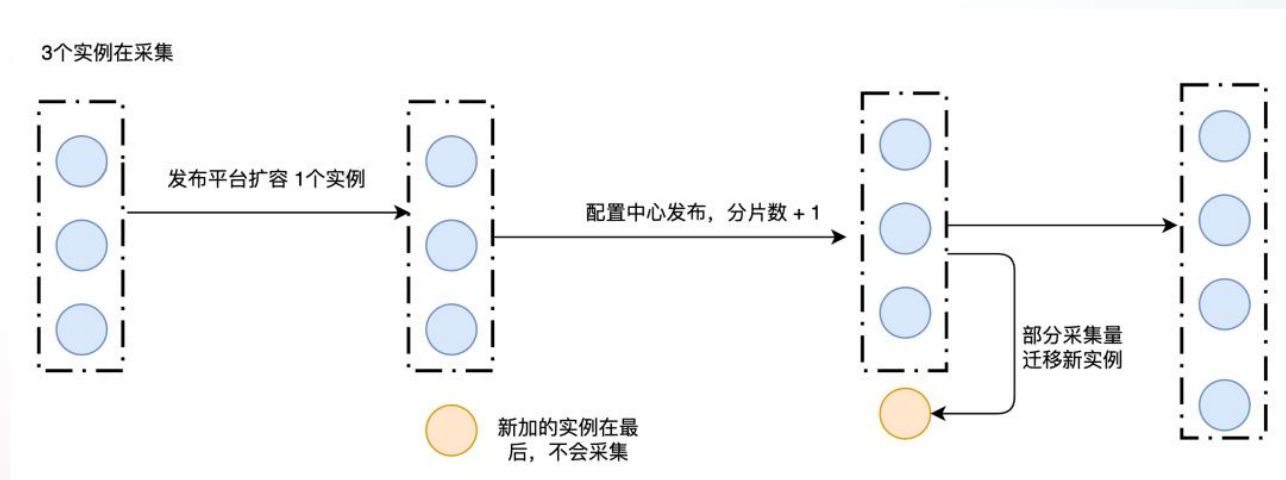
- 移除 Thanos 备用查询链路
- 基于 vmagent 二次开发:
 - 集成配置中心, 规范配置发布
 - 采集保护, 限制异常流量
 - 集中管控 push 指标需求
- 按业务线收敛采集端集群部署



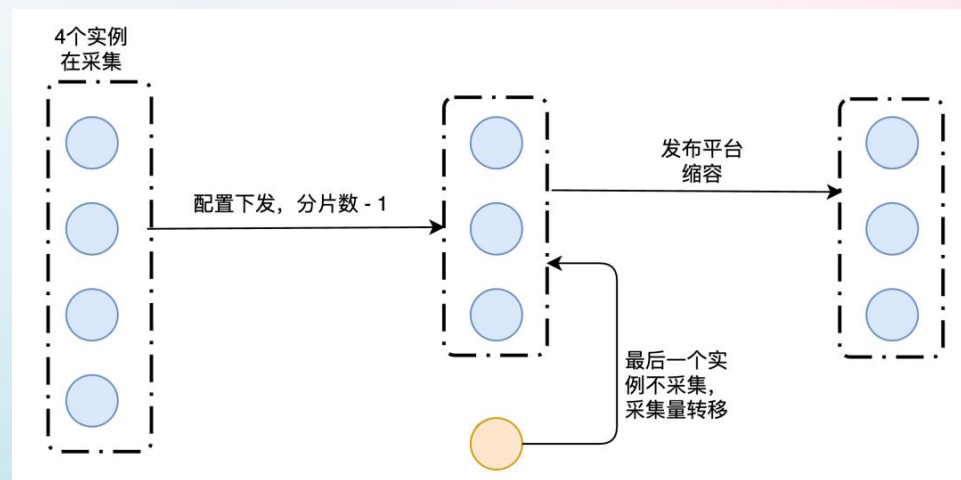
基于 vmagent 开发的采集端

平滑扩缩容

- 采集对象按标签分片，分配给不同实例
- 分片数动态调整，实现无须停机的平滑扩缩容



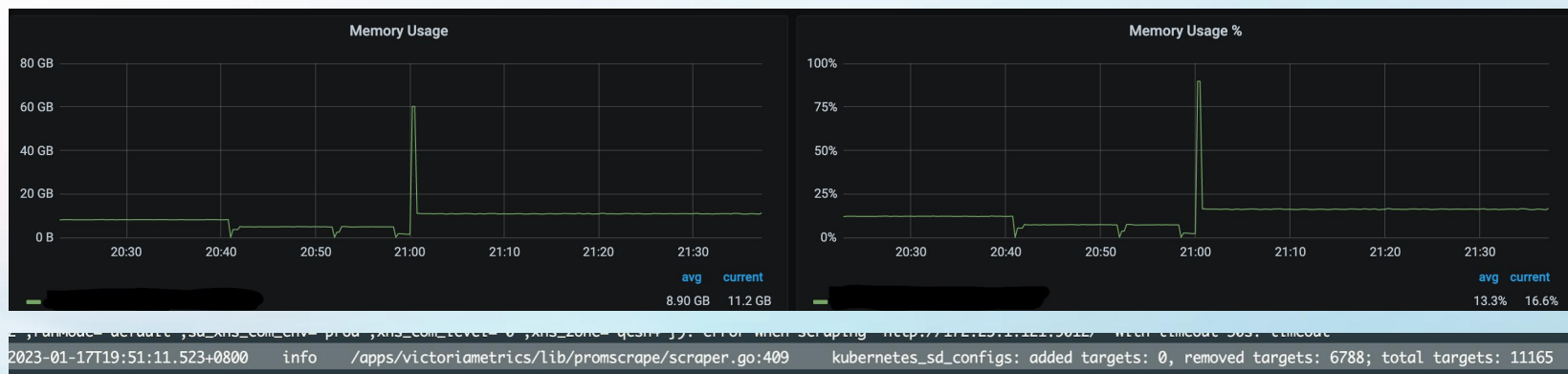
扩容过程



缩容过程

性能优化

- 问题一：重启或新增大量采集对象时，cpu/内存剧烈波动
 - 优化：分批延迟启动采集
- 问题二：删除大量采集对象时，内存飙升/OOM
 - 优化：对象池 + 并发限速
- 问题三：GC 频繁导致的 cpu 利用率高
 - 优化：使用 GOMEMLIMIT 提高 GC 阈值，降低 GC 频率

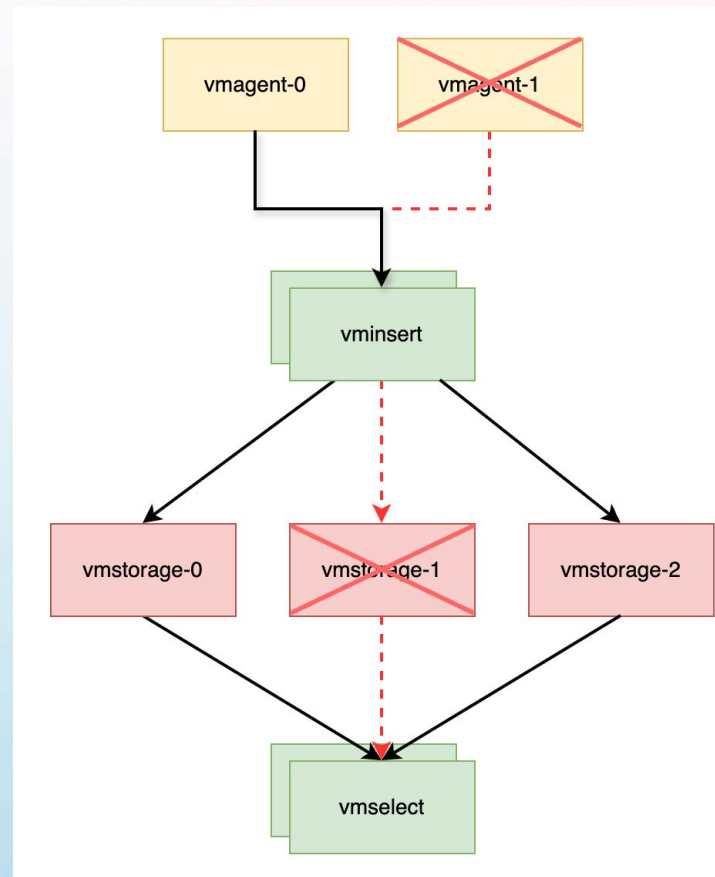


删除大量采集对象内存利用率突增

高可用改造

问题梳理

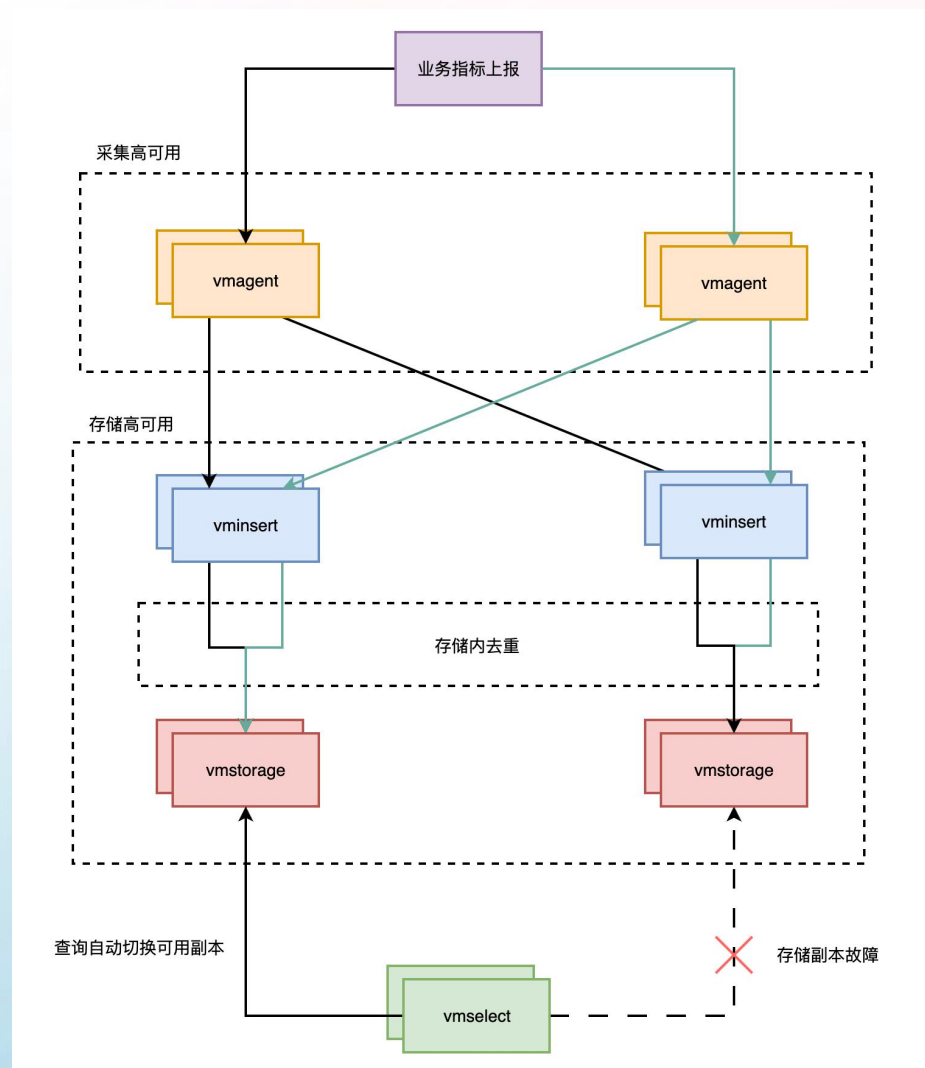
- 采集与存储单副本，无法容忍单实例异常，重启过程中数据异常或整体不可用
- 缺乏服务发现，依赖静态配置更新
- 高负载情况下，雪崩问题频发



采集端或存储端单实例故障

高可用部署

- 采集高可用
 - 双副本冗余采集
 - 存储端去重
- 存储高可用
 - 写入链路双副本
 - 查询自动切换可用存储副本



高可用改造后的部署架构

服务发现

问题:

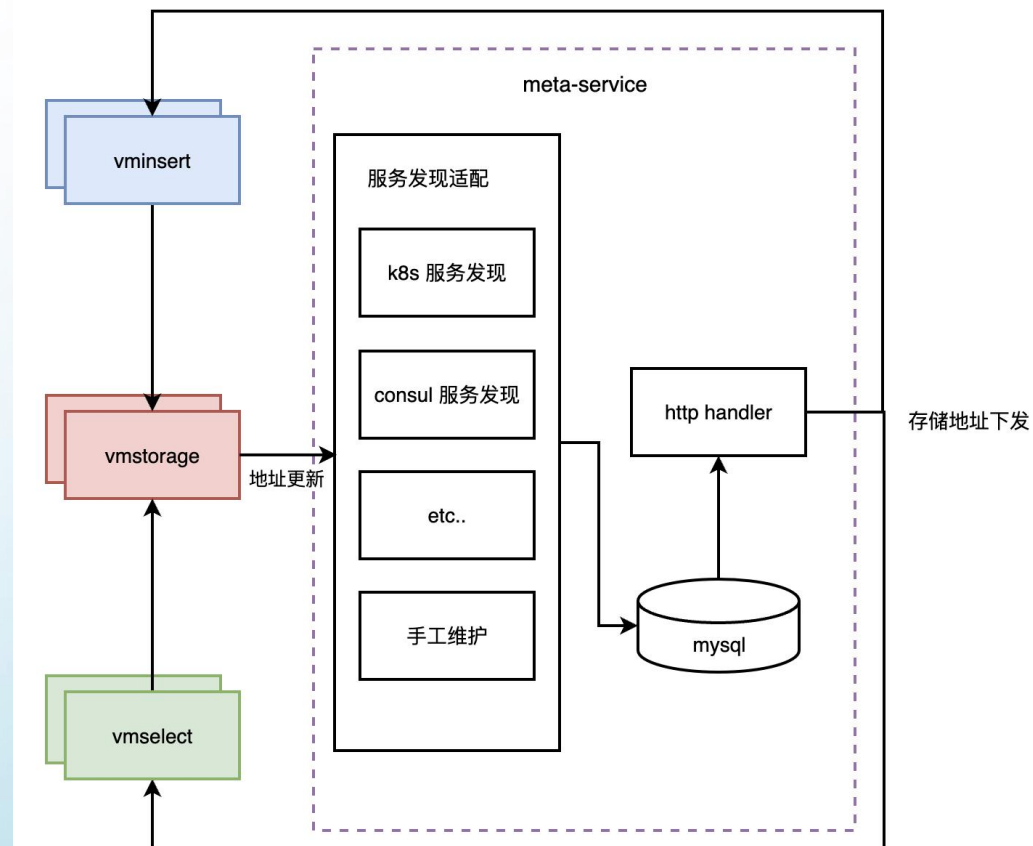
- 静态配置地址列表, 维护困难

过渡方案:

- 依赖云厂商提供的固定 ip 能力

最终方案: meta-service

- 适配多种服务发现机制
- 支持降级到手工维护
- 支持查询自适应切换可用存储副本



服务发现: 存储地址更新下发过程

雪崩问题

现象:

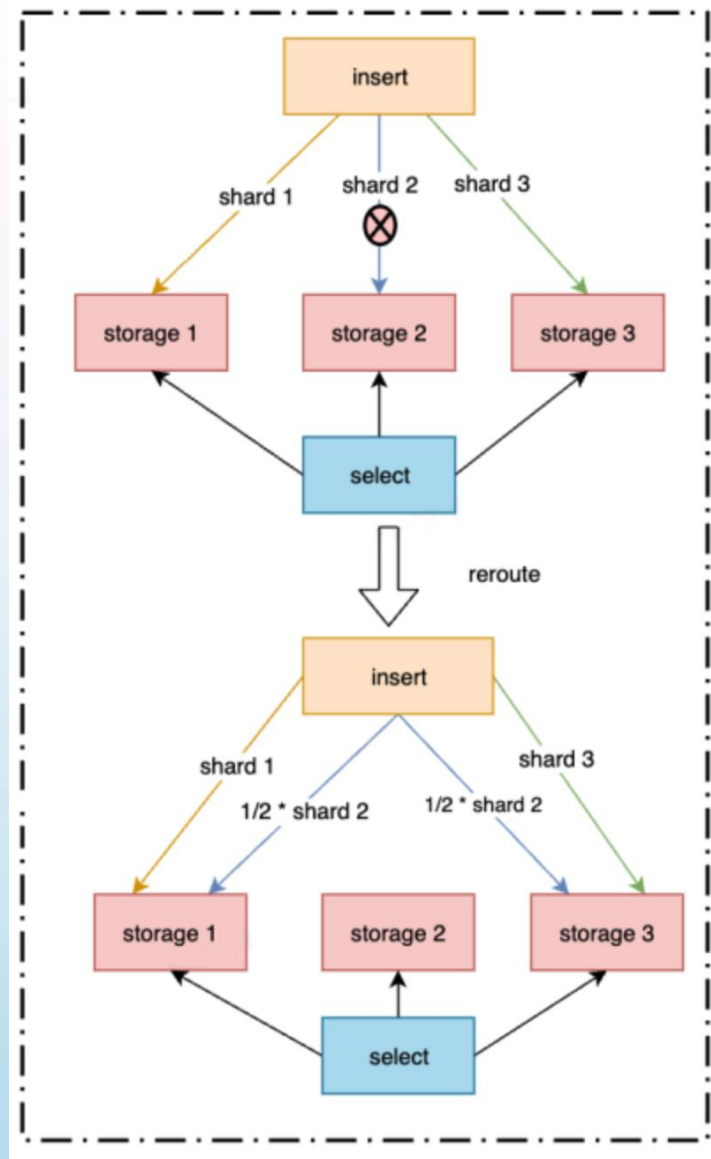
- 某个实例的短暂不可用引发集群吞吐量下降, 无法自愈

根源:

- 较高负载下, 写入重分片(reroute) 机制循环加剧

过程分析:

- 故障触发 reroute: 当部分存储节点不可用时被排除, 写入按剩余节点重新分片
- 重分片加剧负载: 新序列写入带来索引创建的开销, 进一步拖慢更多节点
- 雪崩循环: 被拖慢节点不响应写入, 触发新一轮 reroute



存储节点不可用时 reroute 过程

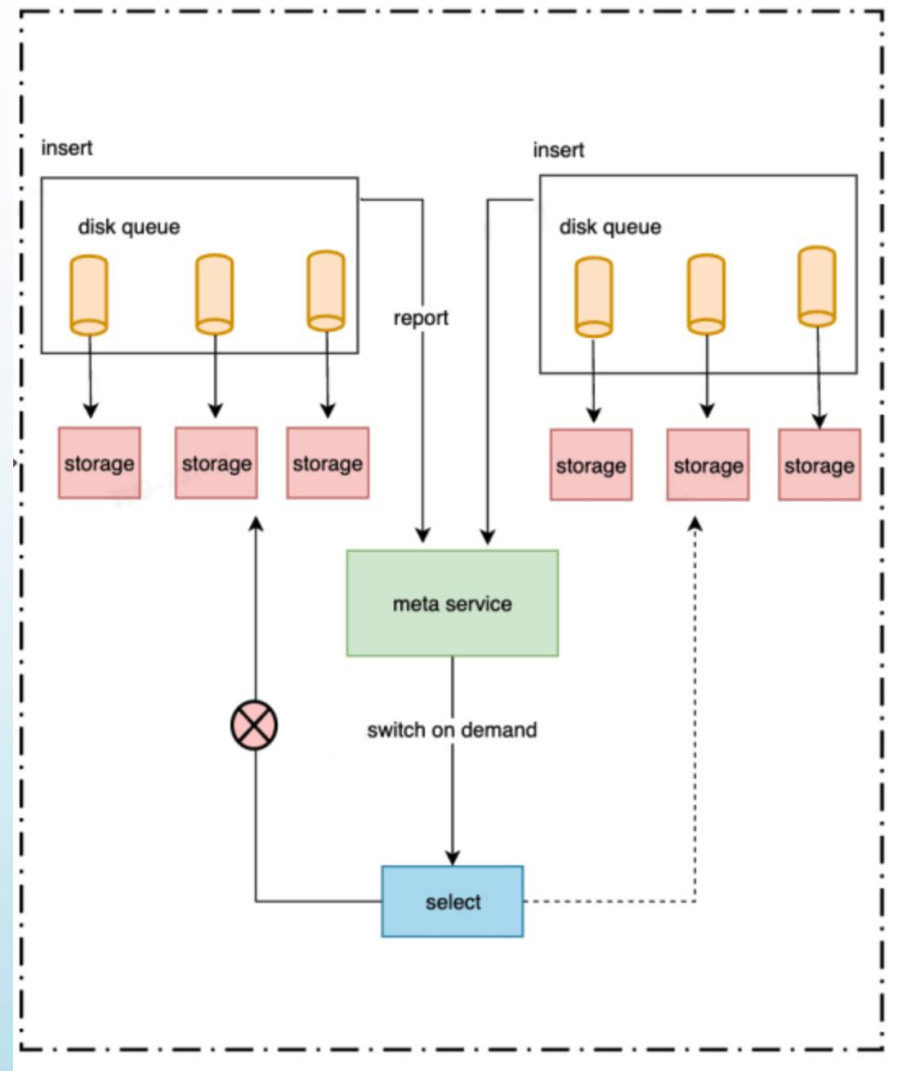
如何解决雪崩

思路:

- 保持写入分片固定, 避免 reroute

方案:

- 写入侧积压代替 reroute, 每个存储节点对应一个磁盘队列文件, 故障时积压, 恢复时回放
- 双副本切换保证查询数据实时性, 切换过程由 meta service 判断自动触发



使用写入侧积压 + 双副本代替 reroute

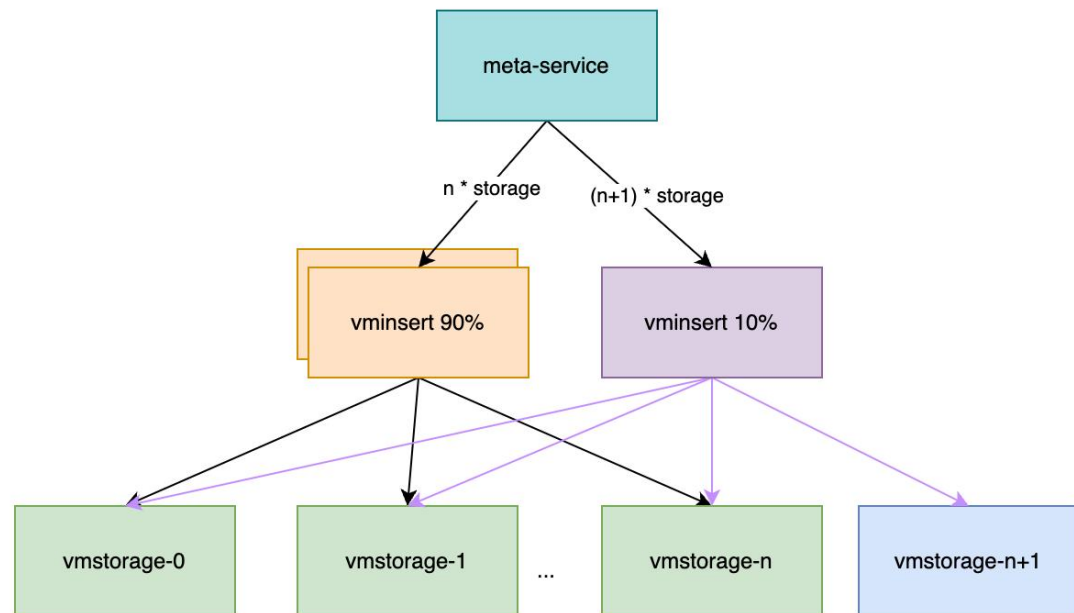
平滑扩缩容

问题:

- 扩缩容时, 如何灰度, 保证存储集群稳定?

方案: 小流量预热索引

- meta-service 选取部分 vminsert 实例下发新的分片配置, 其余实例保持不变
- 确认存储负载稳定后, 全流量生效新的分片配置



灰度生效分片变更

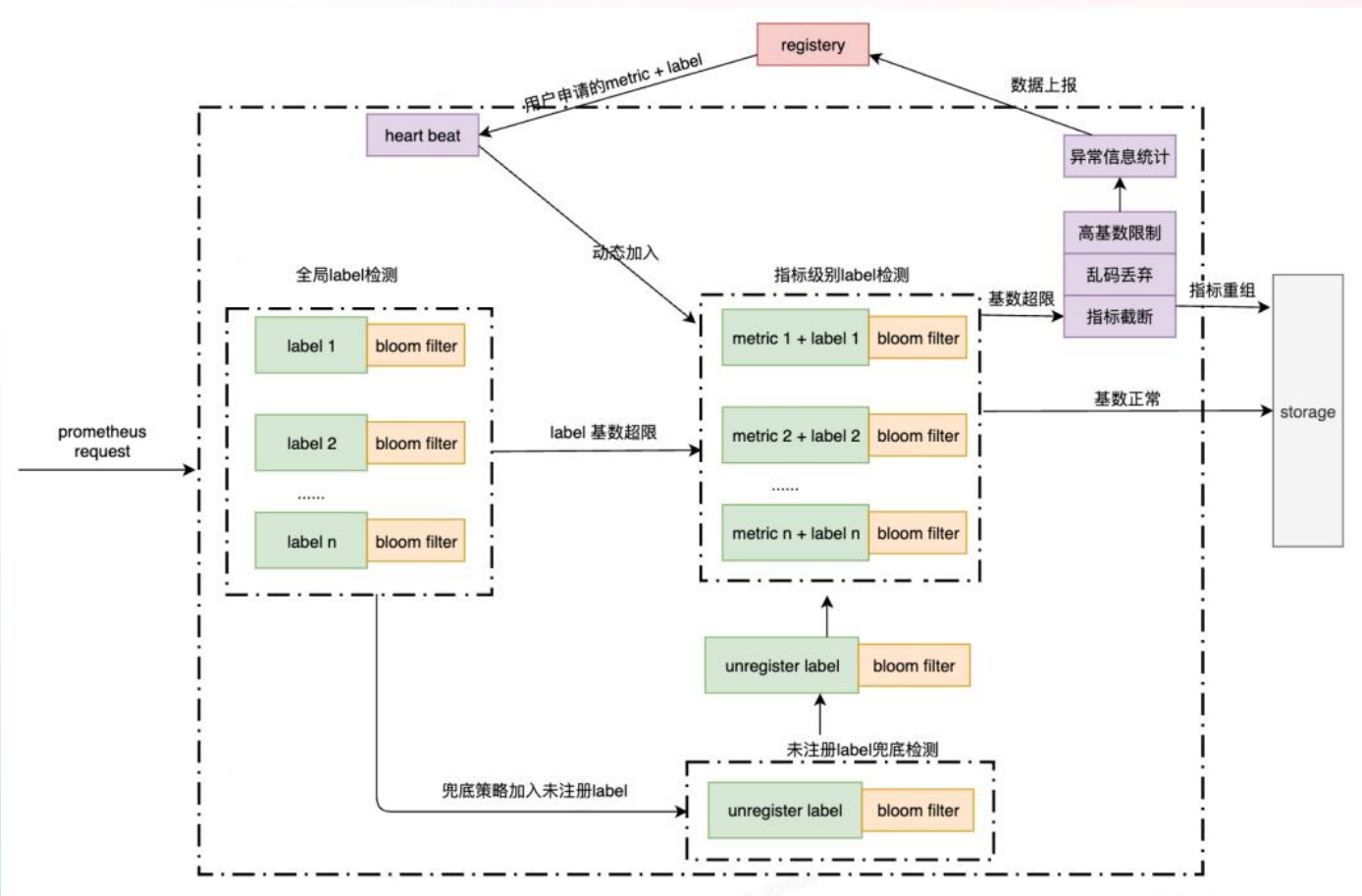
高基数治理

高基数危害:

- 错误的 label 使用姿势导致序列数激增
- 采集和存储负载异常甚至崩溃

治理措施:

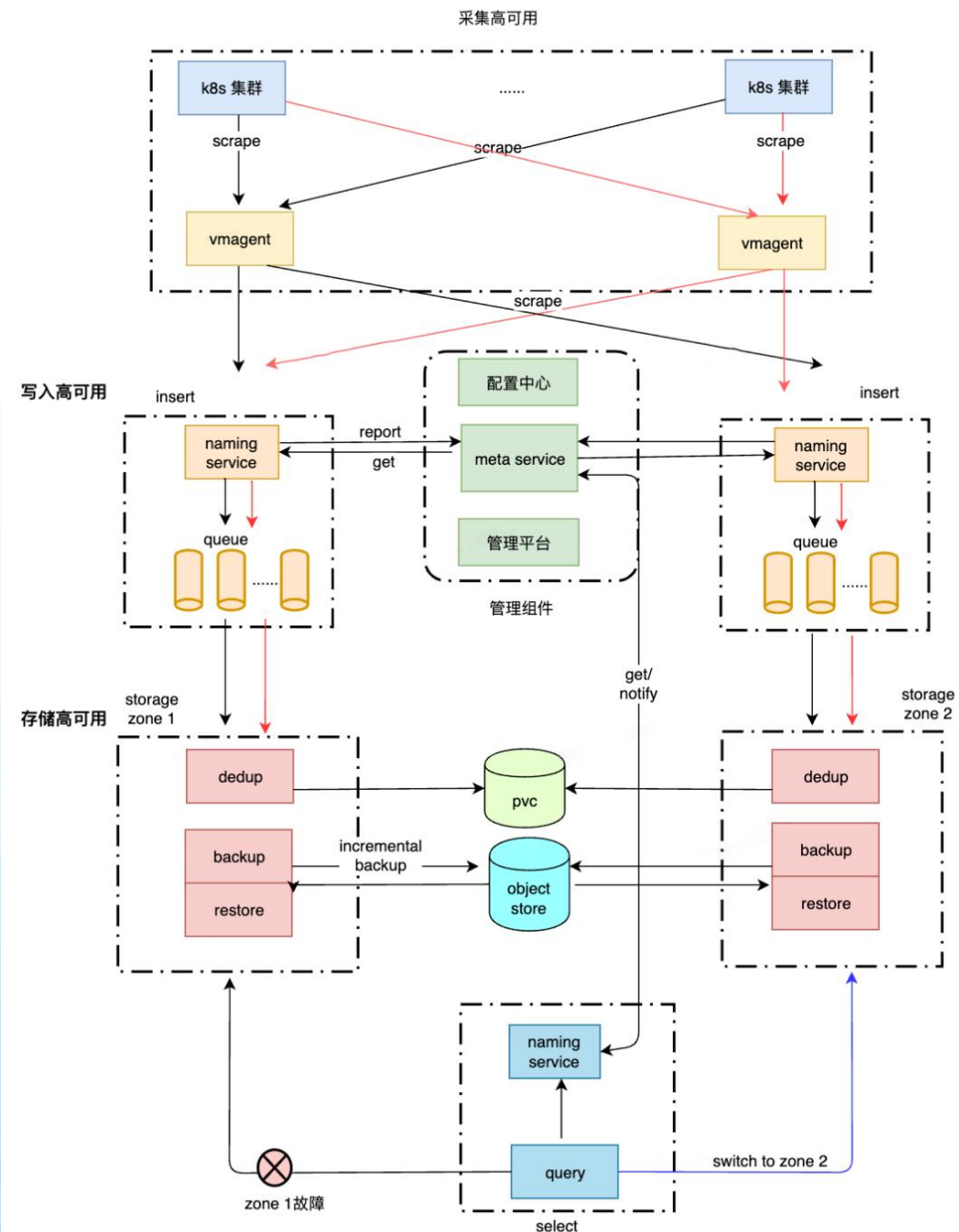
- 默认开启基于 label 基数的管控
- 针对不同场景做全局或者指标级别的基数阈值调整
- 支持按天级月级策略滚动限制基数



写入侧高基数指标管控

高可用改造总结

- 全链路高可用：具备对单实例、多实例故障的容忍能力，局部故障对用户无感
- 雪崩问题治理：集群不可用故障从半年 5次降为 0
- 动态扩缩容：具备分钟级快速扩缩容能力
- 高基数治理：可自动感知和限制高基数指标写入，集群负载降低 15%



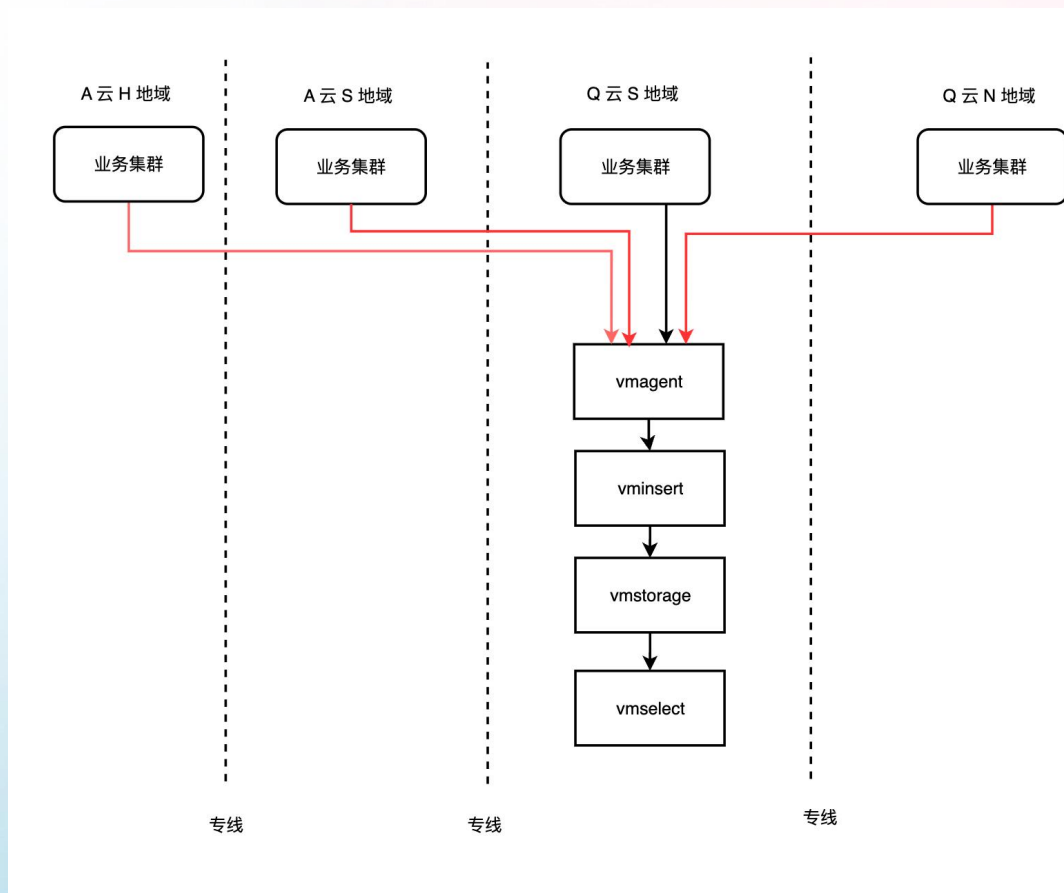
跨云多活部署

背景:

- 单云单地域 -> 多云多地域
- 指标采集经过跨云专线

问题:

- 指标传输带宽大，专线带宽成本极高
- 存在稳定性风险
 - 专线故障，监控丢失
 - 地域级故障，监控整体不可用



单云部署 跨云采集

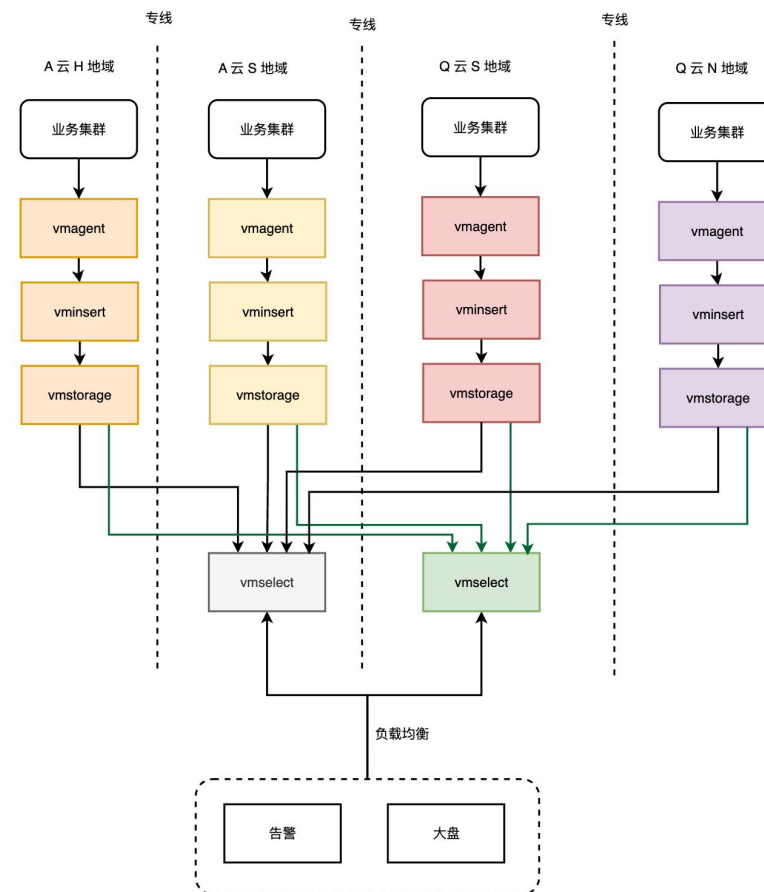
单元化部署

方案:

- 写入链路在各主要云地域单元化部署
- 查询端联合查询聚合所有集群指标, 自动识别并容忍不可用集群

效果:

- 带宽成本降低 80%, 查询流量带宽远小于写入
- 写入链路单元化, 不受跨云专线断网影响
- 地域级别故障不影响其他地域监控

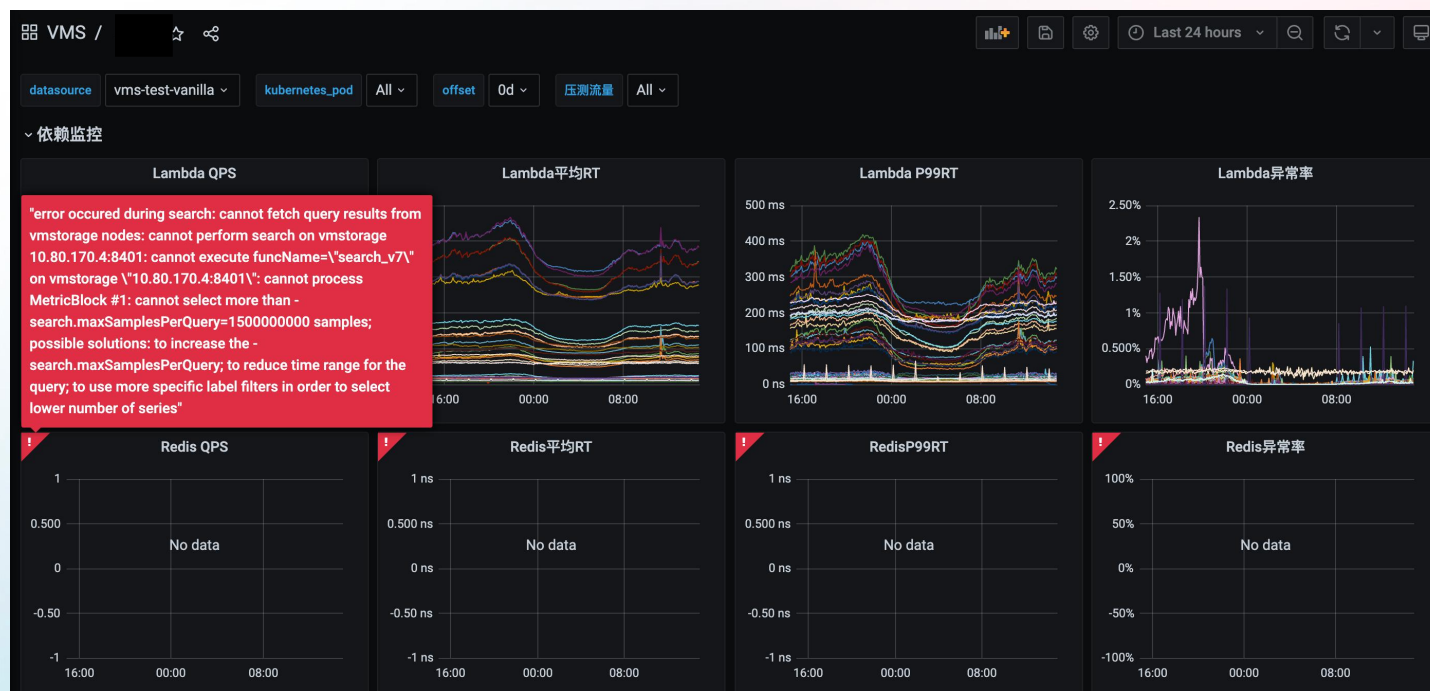


单元化部署 跨云联合查询

查询优化

慢查询案例:

- 推荐业务监控大盘，查询缓慢，大量超时
- 可查询范围小：只能支持最近1h，最近一天范围的查询不可用

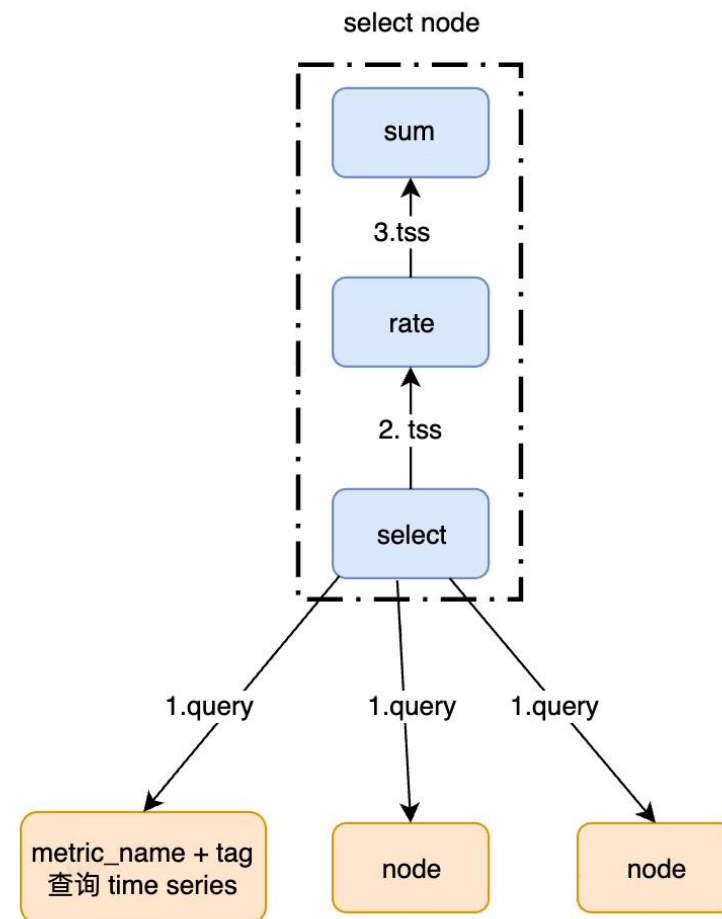


推荐网关指标查询, 无法支持 24h 查询

为什么慢?

问题分析:

- 查询过程: 每个存储节点返回过滤匹配得到的原始指标, 由查询节点完成所有指标的聚合计算
- 指标量大: 小红书推荐等业务场景单业务指标维度已达千万级别
- 瓶颈:
 - 查询节点传输数据量与计算量与指标量成正比
 - 单节点带宽和计算资源有限, 高维度指标查询超时、OOM



常规查询流程

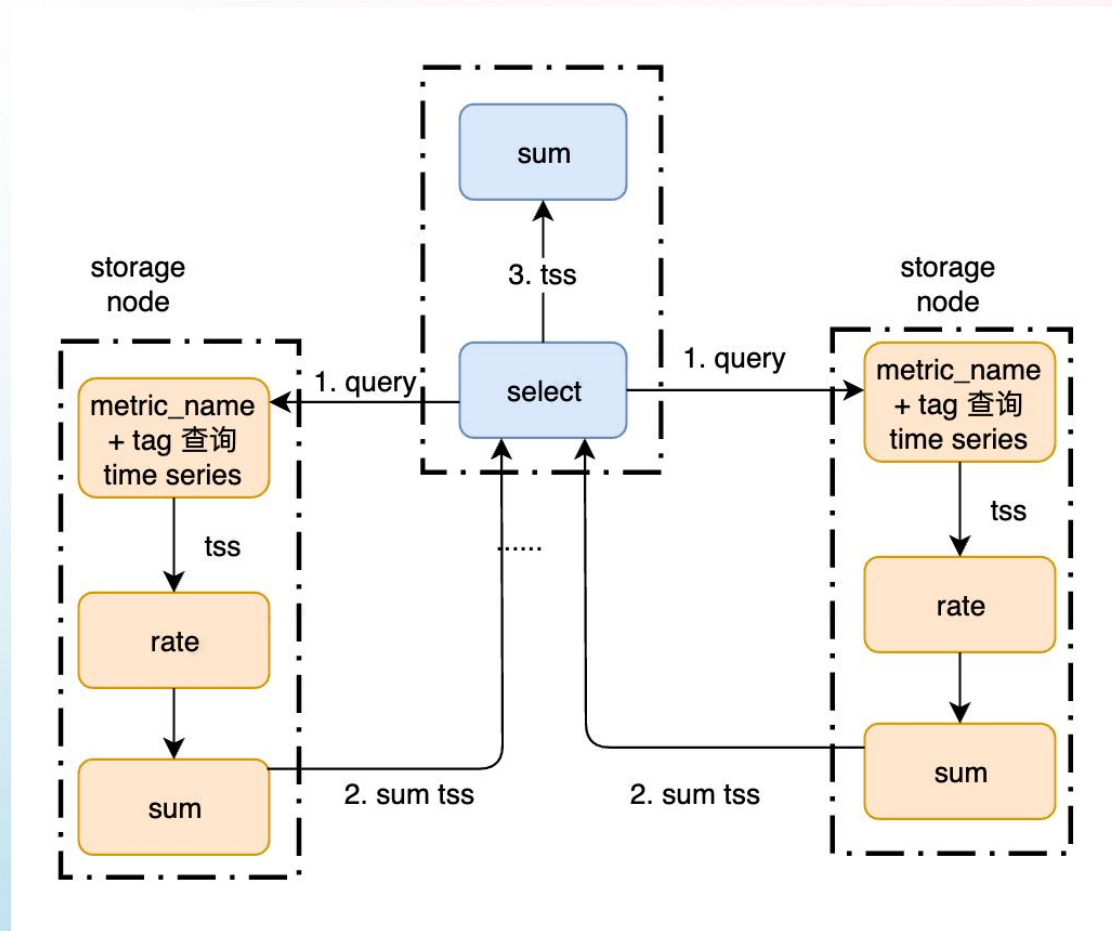
计算下推优化

实现前提:

- 存储节点 cpu 利用率低, 有 buffer
- 高可用改造后, 序列分布稳定, 满足 rate 等窗口函数的下推计算需求

下推过程:

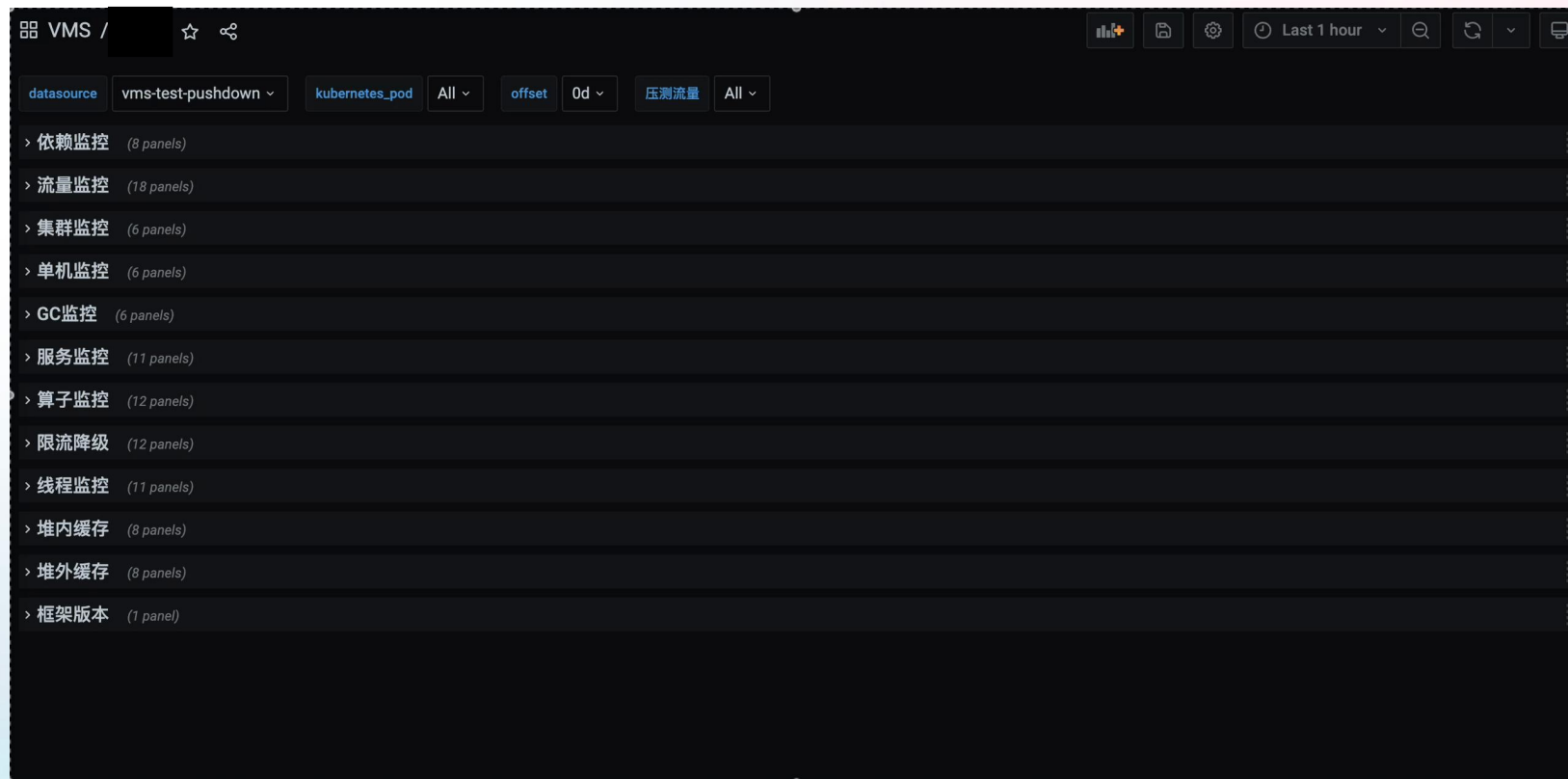
- 存储节点: 对 sum/min/max 等聚合函数预先计算中间结果
- 查询节点: 接收中间结果并进一步聚合
- 特殊处理: avg 与 count 经过特殊处理支持下推



计算下推查询流程

计算下推加速效果

- 查询耗时: 1天内查询降低到 3s, 满足日常巡检需求
- 查询可用范围: 从 1 天提升到 7天

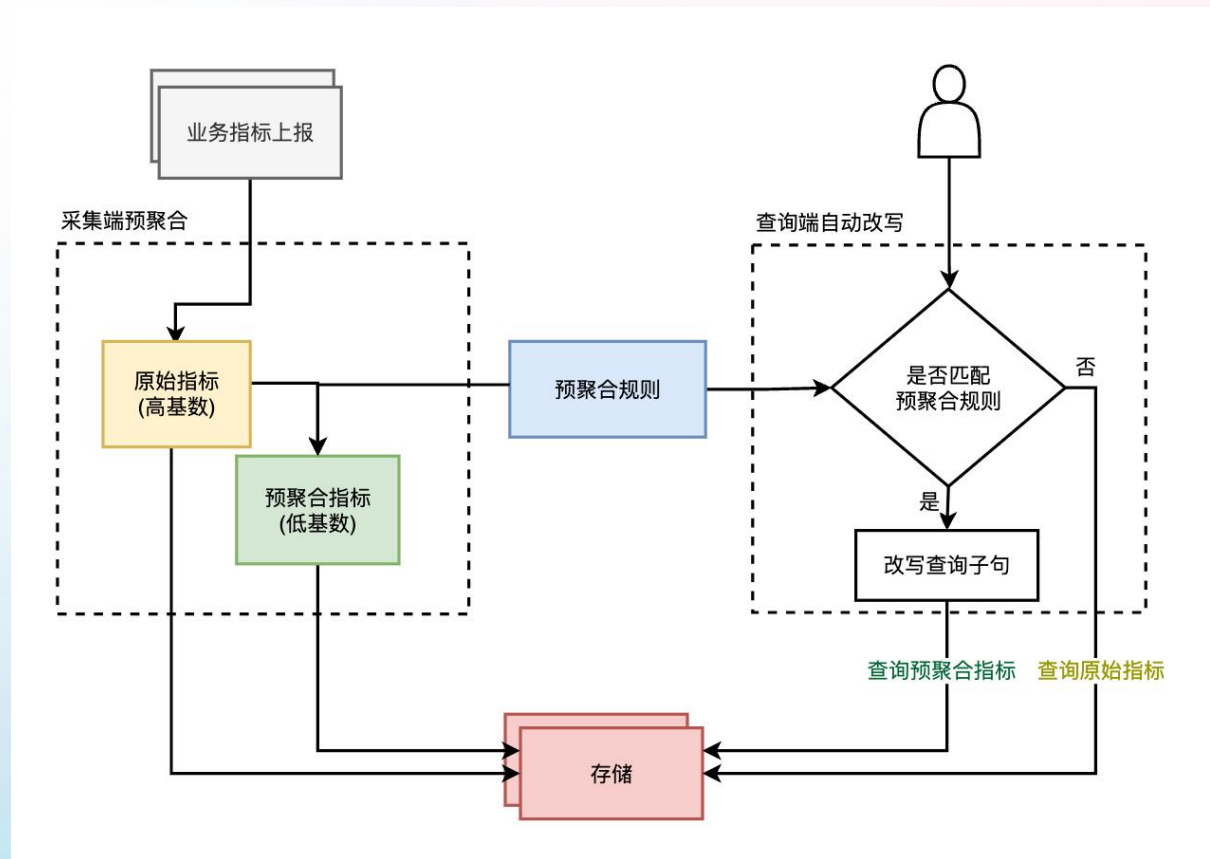


开启计算下推后，可支持7天查询

进一步加速，更高维度的指标查询？

预聚合

- 采集侧按查询需求提前聚合指标
- 查询侧自动匹配并改写查询语句，用户无感加速
- 预聚合配置动态实时生效

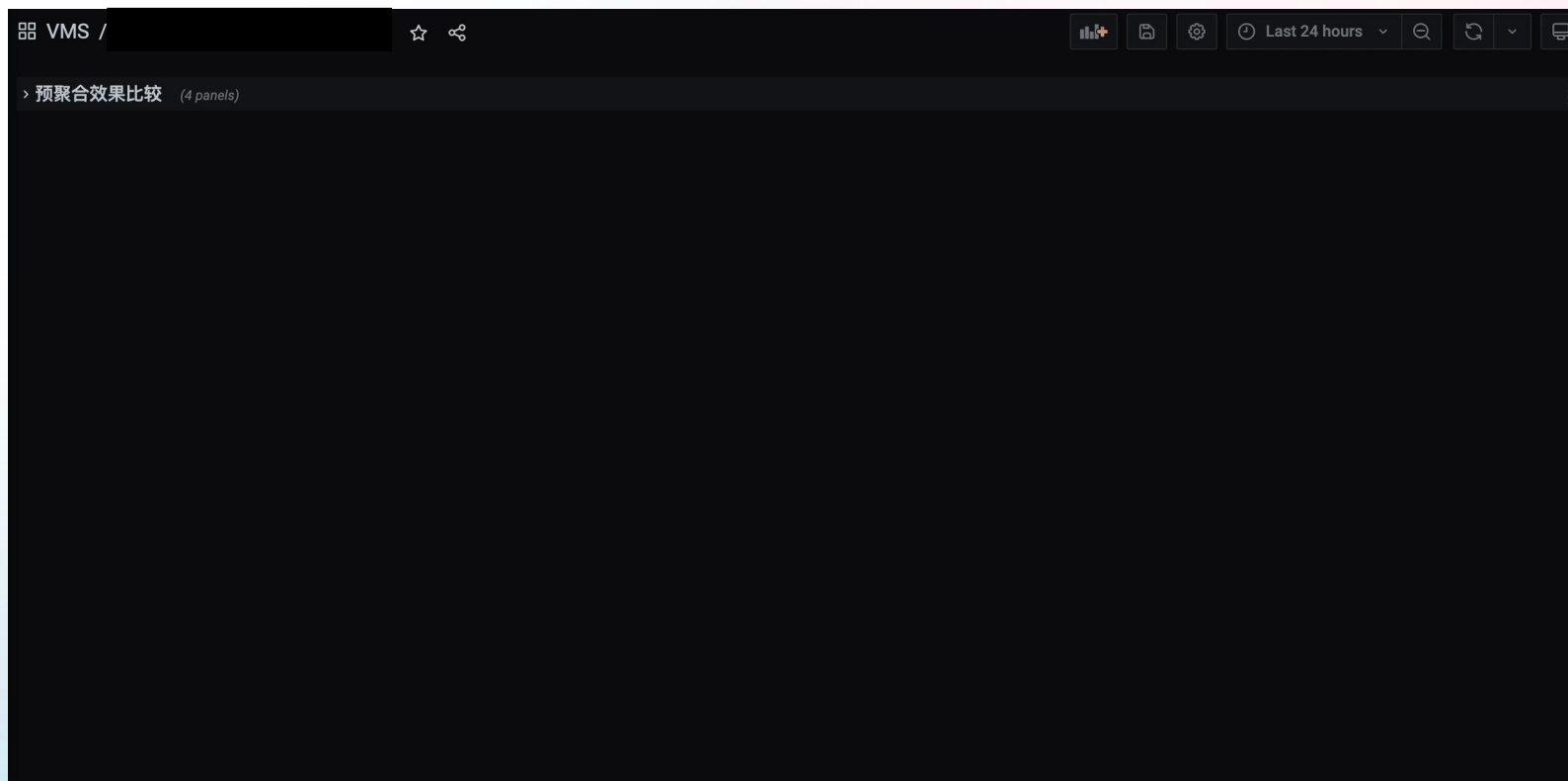


预聚合指标写入与查询流程

预聚合加速效果

以网关 http 请求指标为例

- 查询耗时: 预聚合加速后
所有查询在 1s 内完成
- 查询可用范围: 轻松支持
30天数据的查询



预聚合后的指标可以轻松支持月级别范围查询

总结&展望

稳定性：全链路高可用 + 分钟级弹性扩缩容

性能：采集性能提升近10倍，大查询速度提升数十倍

成本：节省数万核 CPU、数百 TB SSD，带宽成本降80%

运维：实现白屏化操作，日常无需额外运维人力

未来展望：

- 容量运营与自动扩缩容
- 支持离线数据导出